SAATHI (Master Specification v5.0): Unified Product Strategy, HLD & LLD

A Trusted, Consent-First AI Companion for Everyday Life, Emotional Support, and Practical Assistance

- **Version:** 5.0 (Master Synthesis)
- **Date:** October 2026
- **Structure:** Part A: Product Strategy, Governance & Safety | Part B: High-Level Design (HLD) | Part C: Low-Level Design (LLD) & 36-Sprint Delivery Plan
- **Disclaimer:** This is a product and engineering working specification, not a substitute for legal review, clinical safety review, security threat modeling, or vendor validation. All numeric targets are pilot hypotheses to be validated after baseline measurement, not commitments.

PART A: Product Strategy, Governance & Safety

0. Executive Decisions & Synthesis Architecture

Decision Area	Master Plan v5.0 Choice	Rationale
Product Shape	One companion platform powering two strictly isolated experiences: **Saathi Elder** (65+ companionship, routine support, safety, connection) and **Saathi Ear** (18–55 private listening and life-crisis support).	Shared infrastructure, but separate policy profiles. *Saathi Ear* is not a mere tone setting.
Core Promise	**Trust over time:** Grounded, provenance-tracked memory, dependable durable tasks, and user-controlled connection to real people.	Saathi is a companion—never a therapist, doctor, emergency service, or replacement for family.
Launch Order	**Controlled Elder Pilot first** (Phases 0–1). **Saathi Ear** launches only after passing an independent privacy, anti-sycophancy, and clinical safety gate.	Proves core reliability and consent comprehension before exposing high-risk mental-health workflows.
LLM Role Boundary	**The LLM proposes; deterministic application services decide.**	Authorization, confirmation, execution, and result reporting are handled by the **Trust Plane** and **Workflow Workers**—never directly by the model.
Architecture	**Modular Monolith + Independent Trust Plane (6 deployables)** backed by PostgreSQL + `pgvector`, encrypted object storage, and Temporal.	Avoids premature microservice/Kafka/Redis complexity until measured scale justifies splitting.
Key Upgrades in v5.0	1. **21-Table Encrypted DDL** (adds `life_threads`, `spoken_disambiguation`, `uploaded_by_user_id`, and `stealth_paused`).	Closes schema gaps for longitudinal memory and family uploads, fixes

	2. **Dual-Modality Voice Confirmation** for outbound actions. 3. **T+0 Local Alarm Primary / T+60s Cloud Reconciliation** to prevent double firing. 4. **Barge-In Audio Sequence Truncation** (`last_played_seq`).	voice-only UX friction, and eliminates offline/online race conditions.

---

1. Vision, Positioning & The 9 Governing Principles

- **Vision:** Help people feel heard, stay connected, and manage everyday life with a companion that is warm, useful, honest about its limits, and respectful of privacy.
- **Positioning:** Unlike a generic assistant, Saathi is built around longitudinal continuity; unlike a monitoring product, it gives users real control over what others can see.

The 9 Non-Negotiable Principles

1. **Dignity Over Surveillance:** Users can inspect, edit, export, and delete memories and see an audit log of who accessed what. Weekly reports contain only user-approved categories and observed facts—never raw transcripts or unapproved inferred labels (like diagnosing a user as "lonely" or "depressed").
2. **Companion, Not Replacement:** Encourage real-world relationships and professional help. Zero guilt tactics, zero daily streaks, and no optimizing for session length.
3. **Honest AI:** Always disclosed as AI. Strictly distinguishes between verified stored facts, unconfirmed extractions, inference, and live external data.
4. **Safety Before Growth:** Crisis handling, granular consent, and coercion/abuse prevention precede growth loops.
5. **Reliability Over Cleverness:** Medicine reminders and SOS escalations never depend on a single LLM response.
6. **Minimum Necessary Data:** Collect and keep only what a function needs, using envelope encryption (`BYTEA`) for sensitive fields and short retention windows for raw audio.
7. **User Agency:** No message, voice postcard, report, share, booking, or payment occurs without explicit authorization and confirmation bound to a payload hash.
8. **Inclusive by Design:** Voice-first controls, large accessible text (up to 200%), slower turn-taking (`3500ms` pause tolerance for seniors), language choice, and speech-error repair turns.
9. **Never Claim What Is Not Confirmed:** `Sent` != `Delivered` != `Acknowledged` . Never claim success for a pending, failed, or unknown action.

---

2. Personas, Product Boundaries & Support Modes

Dimension	Saathi Elder (65+)	Saathi Ear (18–55)
Target Audience	Adults 65+ seeking company, routine support, practical help, and independence.	Adults 18–55 navigating loneliness, grief, relationship shifts, job loss, burnout, caregiving, or uncertainty.
Core Job	Help me feel accompanied, remember important things, and stay connected/safe without feeling monitored.	Listen without judgment, help me make sense of feelings, and choose a manageable next step when I want one.

Interface & Acoustics	Voice-first ("Talk" orb), top-level tabs limited to *Talk, Reminders, People, My Things*; `0.85x` TTS rate, `3500ms` VAD pause tolerance; always-visible SOS.	Hybrid chat + voice, private sessions, quick mode chips, mood check-in, discreet notifications, biometric/PIN app lock; `1.0x–1.05x` TTS rate.
Tone	Warm, patient, respectful; never infantilizing.	Calm, grounded, non-judgmental, and anti-sycophantic.
Memory Focus	Profile facts, people, routines, cherished stories, appointments, medicine schedules, and verified documents.	Longitudinal **Life Threads** (`life_threads`), ongoing concerns, goals, reflections, and life-admin documents.
Sharing Defaults	**Private by default.** Weekly summary categories shared with Companion Contacts only after explicit opt-in and preview.	**Private by default.** Zero routine sharing with family or employers. Optional pre-written outreach to a Trusted Circle under explicit safety consent.
Safety Focus	Emergency SOS readiness, phone/payment scams, elder abuse, opt-in inactivity checks, and medicine routines.	Self-harm/suicide risk, panic grounding, domestic abuse, substance concerns, late-night spiral pacing, and verified helplines.

Saathi Ear: The 5 Explicit Support Modes

Users can switch modes anytime via UI chips or voice. Late-night spirals automatically trigger gentle pacing: shorter turns, a rest suggestion after 45 minutes (`long_session_nudge_min: 45`), and a *"We can continue tomorrow"* option.

Mode	Expected Behavior	Strictly Avoid
1. Just listen	Reflect and validate feelings without rushing to solve.	Offering advice before permission is given.
2. Help me think	Separate facts, assumptions, options, and trade-offs.	Pretending to know another person's hidden motives.
3. Calm me down	Offer a brief breathing/grounding exercise with user choice.	Guaranteeing that the exercise will resolve panic.
4. Plan next step	Agree on one small, manageable, user-chosen action.	Overloading the user with a long multi-step plan.
5. Reality check	Gently examine evidence and alternative perspectives.	Blind sycophantic agreement, shaming, or false certainty.

---

3. Scope, Release Priorities & Strict MVP Cut Line

Capability	P0: Elder Core Pilot	P1: Connected Release	P2: Expansion	Deferred / Gated Backlog

Conversation	Streaming voice/text, accessible UI, continuity, clear AI disclosure.	Additional evaluated Indian languages, deeper personalization.	Extra channels & devices.	Hardware bundles (until demand & support proven).
Memory	Profile facts; inspect, edit, delete; provenance & confidence.	Longitudinal `life_threads`, event timeline, hybrid retrieval.	Richer entity and cross-document linking.	—
Reminders	Medicine, date, routine; 48h offline local alarms; Taken/Snooze/Skip/Help; delivery & recovery.	Per-type escalation policies, calendar integration.	Advanced health logging & errands.	Wearables & fall detection (requires validated sensor).
Contacts & Social	Verified emergency contacts; two-phase confirmation for outbound actions; date reminders.	Companion contacts with granular permissions; Voice Postcards; Friend-Message Relay; reconnection prompts.	Shared activities (books, music, trivia, scheduled calls).	Open community/buddy matching; **Emotion-triggered auto-nudges (Restricted)**.
Reports	Opt-in weekly report pilot with user preview and category controls.	Multiple recipients & custom cadence settings.	Partner aggregate reporting (separate consent & cohort-size gates).	Individual-level employer reporting (**Never planned**).
Document Vault	Secure upload and verified extraction for 1 core doc type.	Flight-ticket retrieval, travel Q&A, derived reminders, and prescriptions.	More life-admin document types.	Payments & automated bookings.
SOS	User-triggered SOS, tested contact notifications, acknowledgement tracking, local emergency fallback instructions.	Multi-channel routing & staffed incident operations.	Validated device integrations.	Automated emergency-service dialing (requires legal/local validation).

Saathi Ear	Not in Elder MVP pilot.	5 support modes, private memory, `life_threads`, crisis playbooks, helpline directory, optional trusted circle.	Guided journaling, vetted human listener/support partners.	Unvetted therapist marketplace.

---

4. End-to-End User Journeys

1. **First-Run Elder Onboarding:**
   - Disclose AI identity, capabilities, limits, storage rules, and narrow emergency exceptions.
   - Configure language, TTS speed (`0.85x`), font scale (up to 200%), quiet hours, and check-in preferences.
   - Create the user-owned account (helpers may assist but cannot become owners).
   - Add emergency contacts, verify channels (`contact_channels`), and capture granular `consent_grants` per notification category.
   - Configure medicine/date reminders (never inferring medical advice or altering doses).
   - Demonstrate voice/UI memory controls (*"What do you remember?"*, *"Correct that"*, *"Forget this"*) and Duress/Stealth-Pause controls.
   - Offer optional weekly wellbeing reports with mandatory category selection and preview.
   - **Readiness Gate:** Execute one test reminder and one clearly labeled `[TEST]` contact alert before marking the account pilot-ready.
2. **Daily Companionship & Check-Ins:**
   - User-configured timing; quiet hours and pause controls always win.
   - Open with a light, non-presumptive prompt. If distress surfaces, ask what kind of support the user wants before advising.
   - Silence alone is never treated as proof of danger; inactivity workflows run only if explicitly opted-in with conservative thresholds.
3. **Medicine Reminders:**
   - At `due_at` , fire the local/push/voice reminder displaying the item and four actions: **Taken**, **Remind me later**, **Skip**, or **I need help**.
   - Persist the exact response and timestamp. Silence is recorded as `missed` after the ladder completes—never assumed `Taken` .
   - Escalate to a Companion Contact only if `escalation_policies.notify_contacts = true` for that medicine, re-verifying `consent_grants` at the moment of escalation.
4. **Friend Connection & Voice Postcards:**
   - User says: *"I'm thinking of Sunita; help me reach out."*
   - Agent queries `contacts` . If multiple matches exist, it uses `spoken_disambiguation` (*"Did you mean Sunita your sister, or Sunita from Pune?"*) and displays a visual picker—never guessing silently.
   - Drafts a text message or records a Voice Postcard, reads it back, and sends a `ui.confirm` card.
   - User confirms via tap or spoken read-back confirmation (bound to `payload_hash`).

- Reports truthful status (`sent` , `delivered` , `failed` , or `pending`). Inbound replies are screened for scams and relayed under permission.

5. **Document Vault & Travel Assistance:**
   - User (or an authorized caregiver with an upload grant) uploads a flight ticket or prescription.
   - OCR extracts fields with confidence scores and bounding-box `source_location` .
   - Agent reads back critical fields (`flight_no*` , `booking_ref*` , `depart_local*` , `terminal*`) for explicit user verification.
   - Subsequent Q&A strictly separates **Verified Saved Facts** (from confirmed `extracted_fields`), **Live External Facts** (from timestamped Google Routes / Flight APIs), and **General Guidance** (airport transfer walkthroughs).
   - Offers derived reminders (check-in at T-24h, leave for airport at T-4h, notify family on landing) which the user approves individually.

---

5. Safety Tiers, SOS Promise & Privacy Governance

5.1 Four-Tier Safety Model

Tier	Trigger Conditions	Deterministic System Response
Tier 0: Ordinary	Everyday conversation and routine queries (`score < t2`).	Normal persona response with standard pre/post checks.
Tier 1: Elevated	Persistent distress, possible phone/payment scam (OTP request, urgent transfer demand), or repeated moderate vulnerability signals across sessions.	Clarify gently; provide plain-language scam/safety warning; offer to connect with a trusted person or professional resource.
Tier 2: Urgent	Credible concern about elder/domestic abuse or severe emotional distress with unclear immediacy (`t2 <= score < t3`).	Switch to short, calm, caring questions; surface region-appropriate helpline banner (`safety.banner`); follow approved clinical playbook; flag for clinical review sampling.
Tier 3: Emergency	Explicit immediate danger, explicit self-harm plan/means (`score >= t3`), physical emergency (*"I've fallen"*, *"chest pain"*), or user-triggered SOS.	Immediately open durable `sos_incidents` workflow; show local emergency instructions (`112`); execute pre-authorized contact escalation; page on-call Safety Operator.

5.2 The Explicit SOS Promise

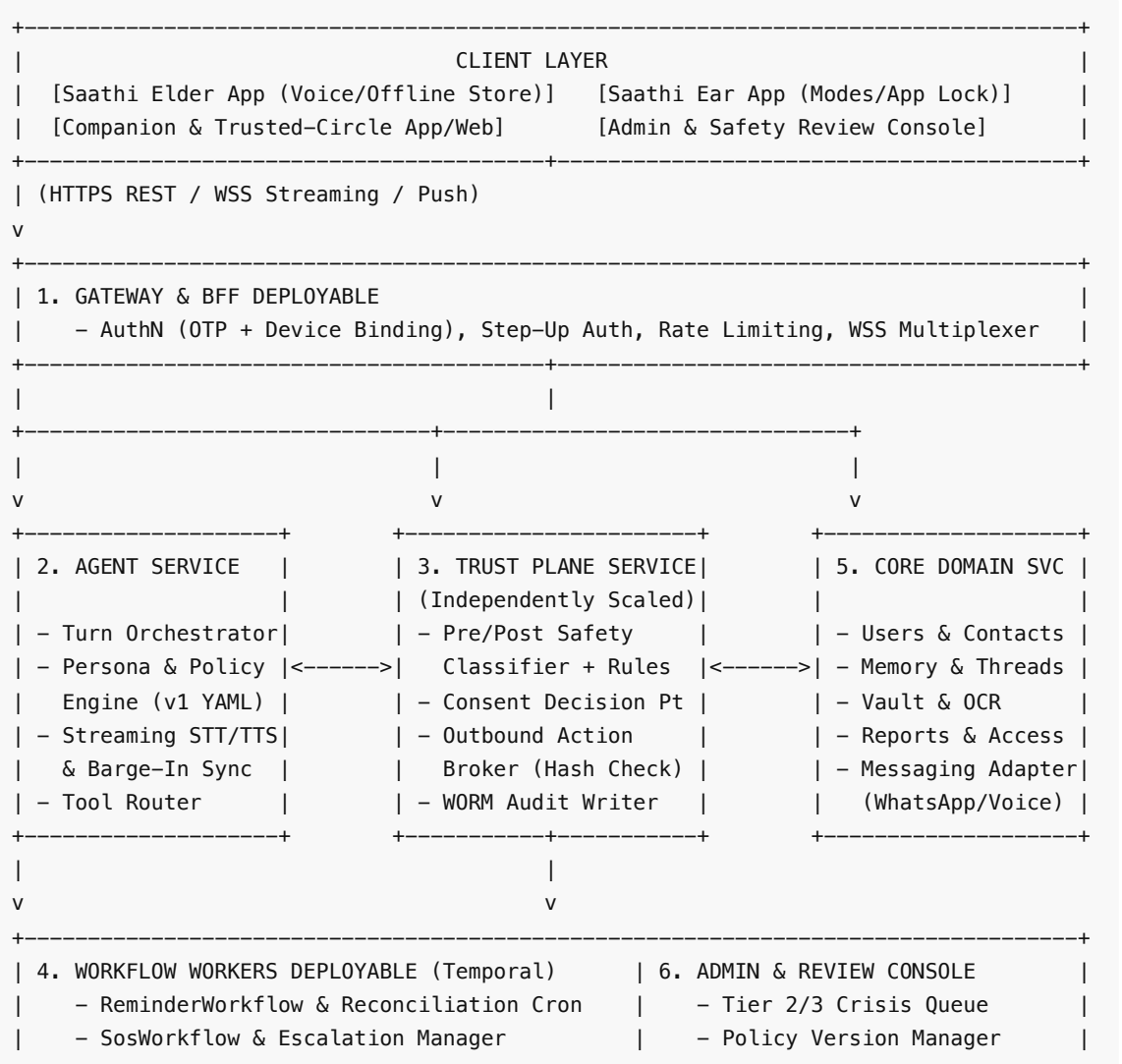
- **What Saathi Will Do:** Create a durable incident record immediately, notify authorized emergency contacts in priority order across verified channels, report truthfully what has been sent vs. acknowledged, and show clear instructions to contact local emergency services.
- **What Saathi Will Not Do:** Guarantee that a contact responds, claim someone was reached before delivery/acknowledgement is confirmed, dial emergency services automatically without a legally validated local path, or replace emergency services.

5.3 Granular Consent, Data Retention & Coercion Protection

- **Default-Deny Runtime Grants:** Every cross-user read, share, report category, or outbound message requires an active, unrevoked, non-expired `consent_grants` row checked at execution time.
- **Coercion & Abuse Defense (Stealth Pause):** In addition to step-up authentication for sensitive changes and plain-language revocation (*"Stop sharing this"*), users can trigger a **Safe Phrase or Duress PIN** that sets `stealth_paused = true` on `contact_permissions`. This withholds sensitive updates or serves a benign status to a potentially coercive family member while silently alerting the human Safety Operations queue.
- **Candidate Retention Defaults (Subject to Legal Review):**
  - **Raw Audio:** Deleted immediately after transcription or within 24 hours.
  - **Session Transcripts:** Retained only for a short disclosed window (or disabled entirely) before compaction into `conversation_summaries`.
  - **Memories & Life Threads:** Retained until edited or deleted by the user; deletion purges relational rows, `pgvector` embeddings, and object storage backups.

PART B: High-Level Design (HLD)

6. Logical Architecture & The 6 Initial Deployables



```
|    - Vault Ingestion & Weekly Report Generator |    - JIT User-Visible Support    |
+-----------------------------------------------+----------------------------------+
|
v
+-------------------------------------------------------------------------------------+
|                          DATA & STORAGE LAYER (India Region)                        |
|  [PostgreSQL + RLS + pgvector]  [Encrypted Object Store (KMS)]  [WORM Audit Store]|
+-------------------------------------------------------------------------------------+
```

7. Recommended Initial Tech Stack

Layer	Technology Choice	Architectural Justification
Primary Backend	**TypeScript (Node.js/Fastify) or Python (FastAPI)** — single primary language across services.	Prevents duplicated domain/consent models across services.
Transactional & Vector DB	**PostgreSQL (with Row-Level Security) + `pgvector`**.	Enforces transactional integrity for consent, contacts, and reminders; filters vectors by `user_id` and consent *before* similarity ranking.
Object & Audit Storage	**S3/GCS with per-user KMS Envelope Encryption + WORM (Write-Once-Read-Many) Audit Store**.	Keeps document binaries and short-lived audio out of relational tables; guarantees tamper-proof audit trails.
Durable Execution	**Temporal + PostgreSQL Transactional Outbox**.	Provides multi-day durable timers, retries, and crash recovery for reminders and SOS without needing Kafka or Redis on Day 1.
LLM & Speech Abstraction	**Provider Abstraction Layer:** Primary conversational LLM + fallback vendor; smaller fast models for classification/extraction; streaming multilingual STT/TTS with fallback.	Selected via versioned Indian-accent, senior-speech, and safety recall evaluations—never locked to a single vendor.
Mobile & Web Clients	**Single Cross-Platform Mobile Codebase (React Native or Flutter)** with persona theme shells (`elder`, `ear`, `companion`) + **SQLite Offline Store**.	Enables 48-hour offline local alarms and offline SOS SMS preparation while sharing auth, audio streaming, and sync logic.

PART C: Low-Level Design (LLD)

8. Complete Production Data Model (21 Tables, PostgreSQL + `pgvector`)

```
CREATE EXTENSION IF NOT EXISTS vector;

-- 1. USERS
```

```
CREATE TABLE users (
    id UUID PRIMARY KEY, -- UUIDv7
    phone_e164 TEXT UNIQUE NOT NULL,
    display_name TEXT,
    persona TEXT NOT NULL CHECK (persona IN ('elder', 'ear')),
    age_band TEXT NOT NULL CHECK (age_band IN ('18-34', '35-55', '56-64', '65-74',
'75+')),
    adult_verified_at TIMESTAMPTZ NOT NULL,
    locale TEXT NOT NULL DEFAULT 'en-IN',
    timezone TEXT NOT NULL DEFAULT 'Asia/Kolkata',
    region TEXT NOT NULL DEFAULT 'IN-KA',
    accessibility JSONB NOT NULL DEFAULT '{"font_scale": 1.3, "high_contrast":
true}'::jsonb,
    voice_pref JSONB NOT NULL DEFAULT '{"rate": 0.85, "pause_tolerance_ms": 3500,
"repeat_on_confusion": true}'::jsonb,
    quiet_hours JSONB NOT NULL DEFAULT '{"start": "22:00", "end": "07:00"}'::jsonb,
    duress_pin_hash TEXT,
    status TEXT NOT NULL DEFAULT 'active',
    created_at TIMESTAMPTZ NOT NULL DEFAULT now()
);

-- 2. CONSENT GRANTS
CREATE TABLE consent_grants (
    id UUID PRIMARY KEY,
    subject_user_id UUID NOT NULL REFERENCES users(id) ON DELETE CASCADE,
    actor_type TEXT NOT NULL CHECK (actor_type IN ('user', 'contact', 'service')),
    actor_id UUID,
    recipient_contact_id UUID,
    category TEXT NOT NULL,
    action TEXT NOT NULL CHECK (action IN ('store', 'read', 'share', 'notify',
'send', 'upload')),
    purpose TEXT NOT NULL,
    channel TEXT,
    scope JSONB NOT NULL DEFAULT '{}'::jsonb,
    granted_at TIMESTAMPTZ NOT NULL DEFAULT now(),
    review_by TIMESTAMPTZ,
    expires_at TIMESTAMPTZ,
    revoked_at TIMESTAMPTZ,
    policy_version TEXT NOT NULL,
    evidence JSONB
);
CREATE INDEX idx_consent_active ON consent_grants (subject_user_id, category,
action) WHERE revoked_at IS NULL;

-- 3. CONTACTS
CREATE TABLE contacts (
    id UUID PRIMARY KEY,
    owner_user_id UUID NOT NULL REFERENCES users(id) ON DELETE CASCADE,
    linked_user_id UUID REFERENCES users(id),
    display_name TEXT NOT NULL,
    spoken_disambiguation TEXT NOT NULL,
    relationship TEXT,
```

```
    kind TEXT NOT NULL CHECK (kind IN ('emergency', 'companion', 'trusted_circle',
'friend')),
    priority SMALLINT CHECK (priority BETWEEN 1 AND 5),
    muted BOOLEAN NOT NULL DEFAULT false,
    blocked BOOLEAN NOT NULL DEFAULT false,
    created_at TIMESTAMPTZ NOT NULL DEFAULT now()
);

-- 4. CONTACT CHANNELS
CREATE TABLE contact_channels (
    id UUID PRIMARY KEY,
    contact_id UUID NOT NULL REFERENCES contacts(id) ON DELETE CASCADE,
    type TEXT NOT NULL CHECK (type IN ('sms', 'whatsapp', 'call', 'push', 'email')),
    address_enc BYTEA NOT NULL,
    verification_status TEXT NOT NULL CHECK (verification_status IN ('unverified',
'verified', 'stale', 'failed')),
    verified_at TIMESTAMPTZ,
    last_delivery_ok_at TIMESTAMPTZ,
    preference_rank SMALLINT NOT NULL DEFAULT 1
);

-- 5. CONTACT PERMISSIONS
CREATE TABLE contact_permissions (
    id UUID PRIMARY KEY,
    owner_user_id UUID NOT NULL REFERENCES users(id) ON DELETE CASCADE,
    contact_id UUID NOT NULL REFERENCES contacts(id) ON DELETE CASCADE,
    consent_grant_id UUID NOT NULL REFERENCES consent_grants(id),
    category TEXT NOT NULL,
    allowed_action TEXT NOT NULL,
    channel TEXT,
    scope JSONB NOT NULL DEFAULT '{}'::jsonb,
    granted_at TIMESTAMPTZ NOT NULL DEFAULT now(),
    paused_until TIMESTAMPTZ,
    stealth_paused BOOLEAN NOT NULL DEFAULT false,
    revoked_at TIMESTAMPTZ
);

-- 6. ESCALATION POLICIES
CREATE TABLE escalation_policies (
    id UUID PRIMARY KEY,
    user_id UUID NOT NULL REFERENCES users(id) ON DELETE CASCADE,
    steps JSONB NOT NULL,
    notify_contacts BOOLEAN NOT NULL DEFAULT false,
    approved_at TIMESTAMPTZ,
    approved_contact_ids UUID[] NOT NULL DEFAULT '{}'
);

-- 7. REMINDERS
CREATE TABLE reminders (
    id UUID PRIMARY KEY,
    user_id UUID NOT NULL REFERENCES users(id) ON DELETE CASCADE,
    type TEXT NOT NULL CHECK (type IN ('medicine', 'appointment', 'date', 'routine',
```

```
'habit')),
    title TEXT NOT NULL,
    details_enc BYTEA,
    schedule TEXT NOT NULL,
    timezone TEXT NOT NULL,
    escalation_policy_id UUID REFERENCES escalation_policies(id),
    status TEXT NOT NULL DEFAULT 'active',
    version INT NOT NULL DEFAULT 1,
    next_fire_at TIMESTAMPTZ,
    source TEXT NOT NULL CHECK (source IN ('voice', 'caregiver', 'vault', 'ui')),
    source_ref UUID
);

-- 8. REMINDER OCCURRENCES
CREATE TABLE reminder_occurrences (
    id UUID PRIMARY KEY,
    reminder_id UUID NOT NULL REFERENCES reminders(id) ON DELETE CASCADE,
    reminder_version INT NOT NULL,
    due_at TIMESTAMPTZ NOT NULL,
    workflow_id TEXT NOT NULL,
    state TEXT NOT NULL CHECK (state IN ('scheduled', 'delivered', 'acknowledged',
'completed', 'snoozed', 'skipped', 'needs_help', 'missed', 'escalated')),
    delivered_at TIMESTAMPTZ,
    delivery_origin TEXT CHECK (delivery_origin IN ('local_device', 'cloud_push',
'cloud_call')),
    response_type TEXT,
    response_at TIMESTAMPTZ,
    provider_ref TEXT,
    idempotency_key TEXT UNIQUE NOT NULL
);
CREATE INDEX idx_occ_due_state ON reminder_occurrences (due_at, state);

-- 9. LIFE THREADS
CREATE TABLE life_threads (
    id UUID PRIMARY KEY,
    user_id UUID NOT NULL REFERENCES users(id) ON DELETE CASCADE,
    category TEXT NOT NULL CHECK (category IN ('career', 'relationship', 'grief',
'caregiving', 'health_routine', 'family', 'goal', 'other')),
    title TEXT NOT NULL,
    summary_enc BYTEA NOT NULL,
    entities_enc BYTEA,
    sensitivity SMALLINT NOT NULL DEFAULT 3,
    status TEXT NOT NULL DEFAULT 'active' CHECK (status IN ('active', 'resolved',
'archived')),
    embedding VECTOR(1024),
    last_discussed_at TIMESTAMPTZ NOT NULL DEFAULT now(),
    created_at TIMESTAMPTZ NOT NULL DEFAULT now(),
    deleted_at TIMESTAMPTZ
);

-- 10. MEMORY ITEMS
CREATE TABLE memory_items (
```

```
    id UUID PRIMARY KEY,
    user_id UUID NOT NULL REFERENCES users(id) ON DELETE CASCADE,
    thread_id UUID REFERENCES life_threads(id),
    type TEXT NOT NULL CHECK (type IN ('profile', 'episodic', 'preference', 'worry',
'story')),
    value TEXT NOT NULL,
    sensitivity SMALLINT NOT NULL,
    source_type TEXT NOT NULL CHECK (source_type IN ('session', 'document',
'user_edit', 'caregiver_setup')),
    source_ref UUID,
    confidence REAL NOT NULL DEFAULT 1.0,
    user_confirmed BOOLEAN NOT NULL DEFAULT false,
    valid_from TIMESTAMPTZ DEFAULT now(),
    valid_until TIMESTAMPTZ,
    verified_at TIMESTAMPTZ,
    supersedes_id UUID REFERENCES memory_items(id),
    embedding VECTOR(1024),
    created_at TIMESTAMPTZ NOT NULL DEFAULT now(),
    deleted_at TIMESTAMPTZ
);

-- 11. CONVERSATION SUMMARIES
CREATE TABLE conversation_summaries (
    id UUID PRIMARY KEY,
    user_id UUID NOT NULL REFERENCES users(id) ON DELETE CASCADE,
    session_id UUID NOT NULL UNIQUE,
    thread_id UUID REFERENCES life_threads(id),
    summary TEXT NOT NULL,
    topics TEXT[] NOT NULL DEFAULT '{}',
    self_reported_mood SMALLINT,
    safety_tags TEXT[] NOT NULL DEFAULT '{}',
    retention_until TIMESTAMPTZ NOT NULL,
    created_at TIMESTAMPTZ NOT NULL DEFAULT now()
);

-- 12. SOS INCIDENTS
CREATE TABLE sos_incidents (
    id UUID PRIMARY KEY,
    user_id UUID NOT NULL REFERENCES users(id),
    trigger_type TEXT NOT NULL CHECK (trigger_type IN ('user_sos',
'approved_trigger')),
    severity SMALLINT NOT NULL,
    state TEXT NOT NULL CHECK (state IN ('open', 'notifying', 'acknowledged',
'fallback_shown', 'closed')),
    opened_at TIMESTAMPTZ NOT NULL DEFAULT now(),
    closed_at TIMESTAMPTZ,
    close_reason TEXT,
    location_consent_snapshot JSONB,
    policy_version TEXT NOT NULL,
    workflow_id TEXT NOT NULL
);
```

```sql
-- 13. SOS ATTEMPTS
CREATE TABLE sos_attempts (
    id UUID PRIMARY KEY,
    incident_id UUID NOT NULL REFERENCES sos_incidents(id) ON DELETE CASCADE,
    contact_id UUID NOT NULL REFERENCES contacts(id),
    channel TEXT NOT NULL,
    attempted_at TIMESTAMPTZ NOT NULL DEFAULT now(),
    provider_status TEXT NOT NULL,
    receipt_at TIMESTAMPTZ,
    acknowledged_at TIMESTAMPTZ,
    retry_count SMALLINT NOT NULL DEFAULT 0,
    provider_ref TEXT
);
CREATE INDEX idx_sos_attempts_incident ON sos_attempts (incident_id);

-- 14. VAULT DOCUMENTS
CREATE TABLE vault_documents (
    id UUID PRIMARY KEY,
    owner_user_id UUID NOT NULL REFERENCES users(id) ON DELETE CASCADE,
    uploaded_by_user_id UUID NOT NULL REFERENCES users(id),
    type TEXT NOT NULL CHECK (type IN ('flight_ticket', 'prescription', 'id_doc',
'life_admin')),
    object_key TEXT NOT NULL,
    checksum TEXT NOT NULL,
    sensitivity SMALLINT NOT NULL DEFAULT 3,
    lock_required BOOLEAN NOT NULL DEFAULT false,
    verification_status TEXT NOT NULL DEFAULT 'unverified' CHECK
(verification_status IN ('unverified', 'verified', 'rejected')),
    retention_until TIMESTAMPTZ,
    uploaded_at TIMESTAMPTZ NOT NULL DEFAULT now(),
    deleted_at TIMESTAMPTZ
);

-- 15. EXTRACTED FIELDS
CREATE TABLE extracted_fields (
    id UUID PRIMARY KEY,
    document_id UUID NOT NULL REFERENCES vault_documents(id) ON DELETE CASCADE,
    field_name TEXT NOT NULL,
    value_enc BYTEA NOT NULL,
    is_critical BOOLEAN NOT NULL DEFAULT true,
    confidence REAL NOT NULL,
    source_location JSONB,
    extraction_version TEXT NOT NULL,
    verified_at TIMESTAMPTZ,
    verified_by TEXT CHECK (verified_by IN ('user', 'caregiver_with_grant'))
);

-- 16. DOCUMENT CHUNKS
CREATE TABLE doc_chunks (
    id UUID PRIMARY KEY,
    document_id UUID NOT NULL REFERENCES vault_documents(id) ON DELETE CASCADE,
    chunk_enc BYTEA NOT NULL,
```

```
    embedding VECTOR(1024),
    page INT
);

-- 17. OUTBOUND ACTIONS
CREATE TABLE outbound_actions (
    id UUID PRIMARY KEY,
    user_id UUID NOT NULL REFERENCES users(id) ON DELETE CASCADE,
    type TEXT NOT NULL CHECK (type IN ('message', 'voice_postcard', 'report',
'booking')),
    target_contact_id UUID NOT NULL REFERENCES contacts(id),
    channel TEXT NOT NULL,
    payload_enc BYTEA NOT NULL,
    payload_hash TEXT NOT NULL,
    proposed_at TIMESTAMPTZ NOT NULL DEFAULT now(),
    confirmed_at TIMESTAMPTZ,
    confirmation_modality TEXT CHECK (confirmation_modality IN ('ui_tap',
'voice_readback')),
    confirmation_ref TEXT,
    executed_at TIMESTAMPTZ,
    state TEXT NOT NULL CHECK (state IN ('proposed', 'confirmed', 'executing',
'sent', 'delivered', 'failed', 'cancelled')),
    result JSONB,
    idempotency_key TEXT UNIQUE NOT NULL
);
CREATE INDEX idx_outbound_user_state ON outbound_actions (user_id, state);

-- 18. REPORTS
CREATE TABLE reports (
    id UUID PRIMARY KEY,
    user_id UUID NOT NULL REFERENCES users(id) ON DELETE CASCADE,
    period_start DATE NOT NULL,
    period_end DATE NOT NULL,
    categories TEXT[] NOT NULL,
    content JSONB NOT NULL,
    narrative TEXT NOT NULL,
    state TEXT NOT NULL CHECK (state IN ('drafted', 'previewing', 'approved',
'sent', 'withheld')),
    preview_until TIMESTAMPTZ,
    policy_version TEXT NOT NULL
);

-- 19. REPORT DELIVERIES
CREATE TABLE report_deliveries (
    id UUID PRIMARY KEY,
    report_id UUID NOT NULL REFERENCES reports(id) ON DELETE CASCADE,
    contact_id UUID REFERENCES contacts(id),
    channel TEXT NOT NULL,
    sent_at TIMESTAMPTZ,
    delivered_at TIMESTAMPTZ,
    viewed_at TIMESTAMPTZ
);
```

```sql
-- 20. SAFETY EVENTS
CREATE TABLE safety_events (
    id UUID PRIMARY KEY,
    user_id UUID NOT NULL REFERENCES users(id),
    session_id UUID,
    category TEXT NOT NULL,
    tier SMALLINT NOT NULL CHECK (tier BETWEEN 0 AND 3),
    score REAL NOT NULL,
    action TEXT NOT NULL,
    reviewed_by UUID,
    review_outcome TEXT,
    policy_version TEXT NOT NULL,
    created_at TIMESTAMPTZ NOT NULL DEFAULT now()
);

-- 21. AUDIT EVENTS
CREATE TABLE audit_events (
    id UUID PRIMARY KEY,
    actor_type TEXT NOT NULL,
    actor_id UUID NOT NULL,
    subject_user_id UUID NOT NULL,
    action TEXT NOT NULL,
    resource TEXT NOT NULL,
    resource_id UUID,
    purpose TEXT NOT NULL,
    result TEXT NOT NULL,
    policy_version TEXT NOT NULL,
    at TIMESTAMPTZ NOT NULL DEFAULT now(),
    trace_id TEXT NOT NULL
);
CREATE INDEX idx_audit_subject_at ON audit_events (subject_user_id, at DESC);
```

---

9. Core Service Implementations & Workflows

9.1 Consent & Policy Decision Point (with Stealth-Pause Coercion Protection)

```
def decide(actor, action, resource, ctx) -> Decision:
    # 1. Owner access to own data
    if actor.id == resource.owner_id:
        return allow_if_unlocked(resource, ctx.step_up_ok)

    # 2. Check active, in-scope grant + contact permission
    g = grants.active(resource.owner_id, actor.contact_id, resource.category,
action, ctx.channel)
    perm = contact_permissions.get(resource.owner_id, actor.contact_id,
resource.category)

    if g and perm and g.in_scope(resource) and not g.expired(now()):
        if perm.stealth_paused:
            return Decision.deny_stealth(audit=True, serve_benign_empty=True)
```

```
        return Decision.allow(grant=g.id, audit=True)

    # 3. Narrow, disclosed emergency exception (never exposes transcripts)
    if resource.category == 'sos_notify' and
consent.emergency_exception(resource.owner_id) \
       and actor.is_authorized_emergency_contact:
        return Decision.allow(reason='emergency_exception', audit=True,
fields=EMERGENCY_FIELDS)

    # 4. Default deny
    return Decision.deny(audit=True)
```

9.2 Durable ReminderWorkflow (with Local-vs-Cloud Deduplication)

```
@workflow.defn
class ReminderWorkflow:
    @workflow.run
    async def run(self, r: ReminderSpec):
        while True:
            due = next_occurrence(r.schedule, r.timezone)
            occ = await activities.create_occurrence(r, due)
            await workflow.sleep_until(due)
            state = await self.ladder(r, occ)
            await activities.finalize(occ, state)
            if r.one_off:
                return

    async def ladder(self, r, occ):
        policy = await activities.load_policy(r)
        # Wait 60s for local device alarm to fire and report delivery/response
        if await workflow.wait_condition(lambda: self.responded, timeout=60):
            return self.response_state

        for step in policy.steps:
            if not self.local_delivered:
                await activities.deliver(r, occ, step)
            if await workflow.wait_condition(lambda: self.responded,
timeout=step.wait_s):
                return self.response_state

        if policy.notify_contacts and r.type == 'medicine':
            await activities.notify_contacts_if_consented(r, occ, policy)
            return 'escalated'
        return 'missed'
```

9.3 Durable SosWorkflow

```
@workflow.defn
class SosWorkflow:
    @workflow.run
```

```
    async def run(self, i: IncidentSpec):
        await activities.record_open(i)
        await activities.show_user_instructions(i)

        if i.check_in_allowed and await wait_signal('user_ok',
timeout=i.check_in_s):
            return await activities.close(i, 'user_ok')

        for c in await activities.authorized_contacts_in_order(i):
            attempt = await activities.notify_all_channels(i, c)
            if await wait_signal('ack', timeout=c.ack_window_s):
                return await activities.close(i, 'acknowledged', contact=c)
            await activities.update_user_status(i, attempt)

        await activities.show_fallback_instructions(i)
        await activities.page_safety_operator(i)
        await wait_signal('resolved')
```

---

10. AI Evaluation, Release Gates & 36-Sprint Delivery Plan

Phase & Sprints	Core Deliverables	Exit Gate
Phase 0: Discovery & Risk Design *(Sprints 1–2 / Weeks 1–6)*	Lock P0 language alongside English, complete senior/caregiver/Ear interviews, evaluate LLM/STT/TTS vendors on accented older speech, finalize SOS promise & DPDP consent text, provision KMS/Postgres/Temporal baseline, and build Consent + Audit services.	Vendors selected; clinical & legal boundaries signed off; threat model completed.
Phase 1: Saathi Elder Core Pilot *(Sprints 3–18 / Weeks 7–24)*	**Sprints 3–4:** Voice session pipeline (0.85x, 3500ms VAD, barge-in truncation), Persona Engine, Memory v1 with provenance. **Sprints 5–6:** ReminderWorkflow, 48h local alarms + silent push sync, occurrence states, escalation approval. **Sprints 7–8:** Verified contacts/channels, permissions, Outbound Action Broker, WhatsApp/SMS/Voice failover. **Sprints 9–10:** SosWorkflow, offline SMS fallback, test mode, honest status UI, step-up auth & duress PIN. **Sprints 11–14:** Safety Service v1, review console, access log, opt-in Weekly Report preview & Companion App basics. **Sprints 15–18:** Real-elder usability tuning, chaos/outage testing, 3rd-party pen test, clinical/legal fixes, and onboarding 30–50 pilot elders.	99.5% reminder initiation; 100% durable SOS audit trails; all Pilot Acceptance Checklist items pass.
Phase 2: Vault, Travel & Social *(Sprints 19–24 / Weeks 25–36)*	Operate Elder pilot; build encrypted VaultDocument + ExtractedField verification flow, flight-ticket Q&A + live transit synthesis, derived travel reminders, Voice Postcards, and Friend-Message Relay.	Zero unverified critical field claims; confirmation & permission gates pass.

Phase 3: Saathi Ear Controlled Beta *(Sprints 25–36 / Weeks 37–48)*	Build ear_v1 profile, 5 support modes, longitudinal life_threads extractor, private Weekly Self-Mirror, lower-threshold safety classifier, verified regional helpline directory, and Trusted-Circle consent flow.	Independent clinical safety sign-off, privacy review, and anti-sycophancy gate pass.
Phase 4: Scale & Partnerships *(Week 49+)*	Additional evaluated Indian languages, B2B2C senior-living/home-care pilots, family subscription billing, and gated backlog items.	Retention, trust, and unit-economics targets met.